

VirtCruise Production Handover Guide

Field · Value

Document ID · DOC-010

Version · 0.8.0-draft.1

Product version · v0.8.0 target; accepted release baseline v0.7.0

Status · Draft

Owner · Platform Owner

Reviewer · Operations, DevOps, System Administration, Security, Finance, Support, and Technical leads

Approver · Business Owner

Classification · Confidential — VirtCruise Production Operations

Last reviewed · 2026-08-03

Contents

1. [Purpose](#)
2. [Production overview](#)
3. [Architecture summary](#)
4. [Environment inventory](#)
5. [Infrastructure responsibilities](#)
6. [Release process](#)
7. [Deployment process](#)
8. [Rollback process](#)
9. [Backup strategy](#)
10. [Restore strategy](#)
11. [Disaster recovery](#)
12. [Monitoring](#)
13. [Logging](#)
14. [Health checks](#)
15. [Scheduled tasks](#)
16. [Security](#)
17. [Certificates](#)
18. [DNS](#)
19. [NGINX](#)
20. [PostgreSQL](#)
21. [Application services](#)
22. [Notifications](#)
23. [Manual Finance operations](#)
24. [Support handover](#)
25. [Incident response](#)
26. [Change management](#)
27. [Operational checklists](#)
28. [Known risks](#)
29. [Future improvements](#)
30. [Appendices](#)

1. Purpose

This is the authoritative production handover package for competent Operations, DevOps, System Administrators, Platform Owners, Support Managers, and Technical Leads assuming safe ownership of VirtCruise. It explains operational outcomes, controls, evidence, ownership, maintenance, incident response, and recovery. It is not a software engineering document or developer guide.

Important: This package does not replace DOC-002, DOC-003, DOC-004, DOC-005, DOC-006, DOC-007, DOC-008, or DOC-009. Use those documents for customer, business, Finance, Support, content, lifecycle, desk-reference, and training responsibilities. DOC-007 remains authoritative for application lifecycle states.

This work makes no application, infrastructure, deployment, production, configuration, or test change. Commands shown are controlled procedure examples and require approved access, resolved inventory values, change authority, backups, rollback target, and peer verification before production use.

Handover acceptance outcome

The receiving team accepts ownership only when:

- named primary/backup owners and approved contact routes exist for every production component;
- the restricted environment inventory is complete and independently verified;
- current/previous immutable release identities, checksums, configuration versions, and database schema version are recorded;
- monitoring, logs, alerts, scheduled work, backup status, retention, restore evidence, certificate/DNS ownership, and capacity are visible to authorised operators;
- normal deployment, rollback, incident, backup, restore, and disaster-recovery tabletop exercises are witnessed;
- secrets are transferred through the approved secret manager/process, never this document; and
- open risks, policy-owned values, vendor dependencies, access gaps, and recovery objectives have accountable decisions.

2. Production overview

VirtCruise provides a static customer/staff browser application served by NGINX and an HTTPS API backed by a Spring Boot service and PostgreSQL. The v0.7.0 frontend is an immutable accepted RC4 archive. Production launches in MANUAL_FINANCE: customers receive approved Finance contact information and their reference/currency, while self-service bank details, review creation, and proof upload remain unavailable until a separate approved activation.

Production ownership requires coordinated control of public routing, TLS, static releases, backend service, database/schema, private proof storage, malware scanning, SMTP/notification dispatch, authentication keys/cookies/CORS, monitoring, logging, backups, and incident communication.

Operating principles

1. Observe before changing; preserve the current state and evidence.
2. Use immutable artifacts and atomic activation; never deploy a working tree.
3. Separate frontend rollback, backend rollback, database compatibility, configuration, and data recovery.
4. Prefer reversible changes and prove rollback before deployment.
5. Never reverse successful additive Flyway migrations solely to roll back application binaries.
6. Never put secrets, tokens, proof content, personal data, raw query strings, or production configuration values in tickets, chat, commands captured in documentation, or source control.
7. Use approved monitoring and authoritative records; a customer report or notification does not replace service state.
8. Stop and escalate when an inventory value, authority, backup, recovery objective, or dependency is unknown.

3. Architecture summary

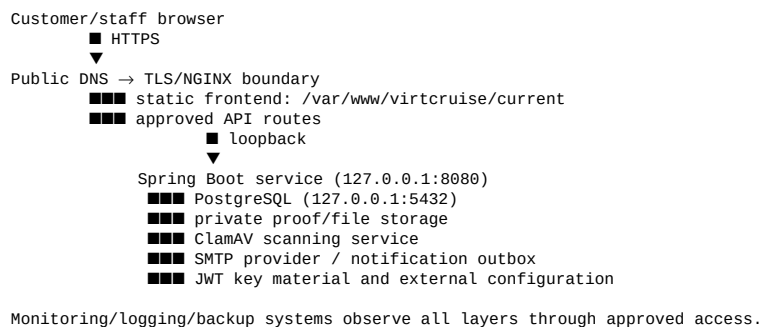


Figure 1 — Production ownership boundary. Public traffic terminates at NGINX; backend/database direct ports are loopback/restricted. Exact hosts, networks, service units, storage roots, and monitoring products belong in the restricted inventory.

Component summary

Component · Accepted role · Primary operational dependency

Frontend · Dependency-free static multi-page browser application · NGINX, immutable artifact, API/TLS/DNS

Backend · Spring Boot 3.5.x API and business workflows · Java runtime/service manager, configuration, PostgreSQL, storage, SMTP, scanner

Database · PostgreSQL 18 accepted architecture; Flyway-managed schema · Durable storage, backup, credentials, capacity, schema compatibility

File/proof storage · Private bank-transfer evidence and metadata linkage · Private root/object availability, permissions, retention, backup, malware scan

SMTP · Verification, reset, financial/booking notification delivery · Protected provider credentials, DNS reputation, outbox/attempt state

ClamAV · Safety scanning for untrusted proof · Scanner availability/signatures, isolated proof processing

Authentication · RS256 JWT, secure HttpOnly refresh cookie, CSRF, exact-origin CORS · Protected key paths, issuer/audience, time, TLS, configuration

Monitoring · Health, service, queue, capacity, certificate, backup, and synthetic signals · Approved monitoring platform and alert routes

Logging/audit · Query-safe operational logs plus immutable business audit · Access control, retention, time synchronisation, search/alert platform

GitHub Pages is a preview artifact, not the customer production origin. Browser mock/local modes are not production recovery paths.

4. Environment inventory

The restricted inventory is the operational source of truth. This guide records accepted public/structural facts and placeholders; it must never hold secrets.

Accepted public and structural inventory

Item · Accepted value/boundary · Verification

Customer frontend · <https://virtcruise.airwide.co.uk> · DNS/TLS/HTTP and approved ownership register

Production API · <https://api.virtcruise.airwide.co.uk> · DNS/TLS/health and approved ownership register

Public health route · <https://api.virtcruise.airwide.co.uk/actuator/health> · Expected bounded response; no sensitive details

Frontend release root · `/var/www/virtcruise/releases/<commit>/` · Host inventory, manifest, permissions

Frontend active link · `/var/www/virtcruise/current` · Resolve and compare approved release

Backend listener · Loopback `127.0.0.1:8080` in accepted topology · Socket/firewall/service evidence

PostgreSQL listener · Loopback `127.0.0.1:5432` in accepted topology · Socket/firewall/database evidence

Backend release convention · `/opt/virtcruise/releases/<version>` evidenced historically · Verify current approved service/unit and path; do not assume

Database/schema · PostgreSQL 18 family; Flyway-managed · Record exact server and applied schema versions

Production payment mode · `MANUAL_FINANCE` at v0.7.0 handover baseline · Authenticated capability plus approved configuration record

Restricted inventory register

Category · Required non-secret fields · Owner · Status/evidence

Servers · Asset ID, role, environment, provider/region, OS, support status, time source · Platform Owner · [complete before acceptance]

Domains/DNS · Zone, registrar/DNS provider, record purpose, TTL, owner, recovery contact · Platform/DNS Owner · [restricted register]

Services · Service unit/container name, version, dependencies, start order, owner · System Administrator · [verified on host]

Ports/network · Source/destination/purpose, exposure, firewall rule owner; no credentials · Security/Platform · [approved network matrix]

Directories · Release/config/log/data/proof/backup roots, owner/group/mode, capacity · System Administrator · [restricted filesystem map]

Certificates · Names, issuer, serial/fingerprint, expiry, renewal method/owner, alert · Certificate Owner · [certificate register]

Storage · Volume/object ID, encryption, capacity, replication, proof isolation, owner · Platform/Security · [storage register]

Backups · Scope, schedule, target, encryption, retention, last success/restore test · Backup Owner · [backup register]

Monitoring · Check/alert name, threshold source, on-call route, runbook, maintenance mode · Operations · [monitoring catalogue]

Vendors · Service, account owner, support route/contract, status page, exit/recovery path · Platform Owner · [vendor register]

Secrets · Secret identifier/purpose, owner, rotation/recovery date—never secret value · Security · [secret manager only]

If any required inventory field is unavailable, mark handover incomplete, assign an owner and due date, and record an interim risk control. Never discover a credential by printing environment files into a terminal transcript.

5. Infrastructure responsibilities

Area · Accountable owner · Operating owner · Required coordination

Platform/servers/network/storage · Platform Owner · DevOps/System Administration · Security, vendor

DNS/TLS/NGINX · Platform Owner · DevOps/System Administration · Security, domain/certificate contacts

Backend service/configuration · Technical Lead · DevOps · Product/Operations, Security

PostgreSQL/schema/backups · Data/Platform Owner · Database Administrator/DevOps · Technical Lead, Security

Proof storage/ClamAV · Security/Platform Owner · DevOps/Security Operations · Finance Operations

SMTP/outbox · Platform/Notification Owner · DevOps/Notification Operations · Support, Security

Authentication/JWT/CORS · Security Owner · DevOps · Technical Lead, Support

Application business operations · Operations Lead · Operations · Finance, Support, Product

Manual Finance · Finance Lead · Finance Operations · Operations, Support

Incident command · Operations Manager · Named Incident Commander · All affected owners

Release/change approval · Business/Change Owner · Release Manager · Platform, Operations, Security, Finance as affected

Customer communication · Support Manager · Support · Incident Commander, Operations

Named people and approved routes belong in the restricted contact register. A backup owner must be able to access required systems without sharing accounts.

6. Release process

Approved candidate → Immutable artifacts/manifests/checksums
→ dependency/config/schema compatibility review → backup/rollback evidence
→ change approval and communication → controlled deployment
→ technical/business acceptance → promote or roll back → close record

Figure 2 — Release governance flow. Release approval, deployment execution, and business acceptance are distinct decisions.

Release preparation

1. Identify approved frontend/backend versions, immutable tags/commits, artifact provenance, bill of materials where supplied, manifests, and SHA-256 checksums.
2. Review release notes, deployment/rollback guides, known limitations, schema migrations, routes, configuration changes, and customer/Finance/Support impact.
3. Confirm the matching backend is healthy and database schema/application compatibility is supported before frontend activation.
4. Resolve all production-owned values through the restricted inventory; verify secrets exist/are readable without displaying them.
5. Confirm recent successful backups and accepted restore-test evidence; define recovery point objective (RPO) and recovery time objective (RTO) from approved policy.
6. Name deployer, peer verifier, approver, acceptance owner, incident commander, and rollback authority.
7. Record change window, customer impact, communication, monitoring maintenance, rollback triggers/target, and go/no-go time.
8. Use [Production Checklists](#); unresolved blocking item means no-go.

7. Deployment process

The detailed execution sequence is in [Operations Runbook RB-01](#). It preserves the accepted pattern: verified immutable artifacts, new release directories, server-side checksum verification, controlled permissions, backend/schema compatibility, NGINX validation, atomic frontend symlink cutover, bounded service action, and acceptance.

Deployment phases

1. **Pre-flight:** confirm authority, participants, backups, current targets, capacity, health baseline, change freeze, and rollback.
2. **Stage:** transfer to a temporary restricted location; verify artifact checksum before extraction; never overwrite an existing release.
3. **Prepare:** create immutable release paths, apply approved ownership/modes, verify manifest and configuration references, and inspect migrations without exposing secrets.
4. **Activate backend where required:** follow the accepted backend release runbook and service unit. Do not invent commands from this frontend repository.
5. **Activate frontend:** switch current atomically only after `nginx -t` succeeds; reload NGINX, not unrelated services.
6. **Verify:** health, TLS/DNS, homepage/assets/packages, authentication, ownership, Manual Finance, Finance/Financial portals, queues/outbox, logs, and resource signals.
7. **Accept:** Technical and business owners record success against release criteria; monitor for the approved observation window.

Do not run a live write transaction merely because an older guide used a synthetic quote. Any production acceptance transaction requires separately approved synthetic-data and cleanup procedures, ownership, and audit evidence.

8. Rollback process

Rollback is a controlled release response, not data restore. Use [Operations Runbook RB-02](#) and the release-specific rollback guide.

Decision and boundaries

- Trigger on predefined severe acceptance failure, customer/security/financial risk, incompatibility, or incident-command decision.
- Record current/previous targets and checksums before change.
- Frontend rollback atomically repoints the current symlink to a verified prior immutable release, validates NGINX, reloads, and rechecks public behavior.
- Backend rollback uses the approved backend runbook/service unit and must remain compatible with applied schema/configuration.
- Do not reverse successful additive Flyway migrations solely for binary rollback.
- Database/data restore is a separate recovery decision requiring scope, RPO, data-loss analysis, Finance/business approval where relevant, and the disaster-recovery process.
- After rollback, confirm authentication, Manual Finance, customer/staff routes, health, outbox/queues, storage, logs, and customer impact; keep the failed release/artifacts/evidence.

Emergency rollback may shorten normal approval only under the approved emergency change policy. It never removes peer verification, incident record, known target, or post-action validation.

9. Backup strategy

Backup design must satisfy approved RPO/RTO, retention, encryption, geography, legal/privacy, and recovery requirements. These values are policy-owned and must be entered in the restricted backup register before handover.

Scope · Minimum backup expectation · Verification

PostgreSQL · Consistent full backup plus approved point-in-time mechanism where required; roles/extensions/schema metadata included appropriately · Automated success, checksum, catalogue, periodic isolated restore and application validation

Frontend/backend artifacts · Immutable released artifacts, manifests, checksums, provenance, release notes · Re-create current/previous release without working tree

Proof/file storage · Private encrypted backup preserving object/metadata linkage, permissions, retention/legal holds · Sample metadata/object reconciliation in isolated restore; never open unsafe proof

Configuration · Versioned non-secret templates and encrypted/controlled production configuration backup · Compare identifiers/permissions; validate without printing values

Secrets/keys · Approved secret-manager backup/recovery and key rotation process · Recovery tabletop with Security; never export into ordinary backup

NGINX/TLS/DNS · Approved config, certificate metadata/renewal config, DNS zone/registrar recovery data · Syntax test and recovery tabletop; secret key protected separately

Monitoring/runbooks · Check definitions, dashboards, alert routes, maintenance procedures · Restore/import validation in approved environment

Audit/logs · Retained according to security/legal policy and protected from alteration · Search/access test and retention verification

Backups must be encrypted in transit/at rest, access-controlled, monitored, protected from the same failure domain where policy requires, and tested. A successful job message is not proof of restorability.

10. Restore strategy

Restore only under an approved incident/change record using [Disaster Recovery Guide](#) and [RB-05](#). Prefer an isolated validation environment before production cutover.

Ordered restore controls

1. Define incident, lost/corrupt scope, target time/version, RPO/RTO, customer/financial/privacy impact, and authority.
2. Quiesce affected writes or isolate the environment where required; preserve evidence.
3. Verify backup identity, completeness, encryption access, checksums, dependency/config/schema compatibility, and clean recovery host.
4. Restore platform/network/DNS/TLS prerequisites and secret references through approved owners.
5. Restore PostgreSQL to the approved recovery point; validate engine, schema/Flyway history, constraints, counts/reconciliation, and access.
6. Restore private proof/files and reconcile metadata/object identity, permissions, quarantine/scan state, retention, and missing objects without treating restored bytes as safe.
7. Restore application releases/configuration, NGINX, monitoring, scheduled tasks, SMTP/scanner integration, and start services in dependency order.
8. Validate health, logs, authentication, ownership, catalogue, Manual Finance, financial totals, review cases, outbox, notifications, proof access controls, and representative read-only business records.
9. Re-enable writes/traffic only after technical, Security, Finance/Operations, and incident-command approval as affected.
10. Monitor, communicate, reconcile the recovery window, preserve evidence, and schedule post-incident review.

11. Disaster recovery

[Disaster Recovery Guide](#) is the authoritative scenario playbook. The recovery priority is safety and control-plane access, public routing/TLS, data integrity, authentication, core application/API, customer/Finance operations, notifications, and full monitoring/normalisation. Approved business impact analysis must assign exact RPO/RTO; no value is invented here.

Detect → Declare/assign command → Contain/preserve → Assess scope and recovery point
→ Recover dependencies/data/services → Validate technically and operationally
→ Controlled traffic/write restoration → Reconcile/communicate → Postmortem

Figure 3 — Disaster-recovery workflow. Recovery may choose rebuild, failover, restore, vendor recovery, or controlled degraded service. Manual Finance remains a configured business mode, not a generic infrastructure fallback.

Covered scenarios include infrastructure loss, database corruption, proof/storage loss, certificate expiry, DNS failure, SMTP failure, and application failure. Cross-region/site failover is not assumed; record actual capability in the restricted inventory.

12. Monitoring

Monitoring must show service availability, correctness, security, capacity, and business-processing risk without exposing customer/proof/secret data.

Signal · Monitor · Example response/runbook

Public frontend/API/TLS/DNS · External HTTPS status, certificate, DNS resolution, latency · RB-03 health triage; certificate/DNS DR scenario

Health endpoint · Bounded /actuator/health availability from approved source · Inspect backend/dependencies; avoid exposing detail publicly

NGINX/backend service · Active state, restart/failure count, upstream errors · RB-03; compare recent change/logs/capacity

PostgreSQL · Availability, connections, locks, replication/PITR if configured, backup, storage · RB-04/RB-05; database owner

Proof storage/ClamAV · Mount/object availability, permissions, capacity, scan queue/failures/signature currency · Stop unsafe viewing; Security/Platform owner

SMTP/outbox · Provider reachability, pending/retry/failed/manual-intervention age/count · RB-06; never blind resend

Scheduler/workers · Last success, duration, missed/overlapping runs, stale claims · RB-07 scheduled-task failure

Disk/memory/CPU · Utilisation/trend, inode/swap/process pressure · Capacity runbook and policy thresholds

Authentication · Login/refresh/CSRF failure trend, key/certificate time validity · Security/Technical Operations; no token logging

Business queues · Review non-terminal/overdue, integration/projection exceptions · Operations/Finance procedure, not direct mutation

Backups · Job success, age, checksum/catalogue, target capacity, restore-test due · RB-04; backup owner

Logs/audit · Ingestion lag, error/security patterns, time sync, retention · RB-08 security/logging escalation

Thresholds, severity, alert routing, on-call hours, deduplication, and maintenance suppression are policy-owned. Every alert catalogue entry needs an owner, actionable condition, dashboard/query, runbook link, safe context, and test evidence.

13. Logging

Accepted access logging records method plus normalised path/URI and omits raw request targets/query strings. Preserve request/correlation identifiers and UTC timestamps. Restrict and retain logs under approved Security/Privacy policy.

Never log or paste:

- passwords, tokens, cookies, CSRF values, private keys, SMTP credentials, environment-file contents;
- raw query strings that may contain identifiers;
- customer payloads, proof filenames/bytes/content, financial DTOs, bank data, recipient lists, or notification bodies;
- unbounded exception objects or headers containing authentication; or
- speculative fraud/security conclusions.

During an incident, record component, normalised route/action, safe status/error category, time, request ID, release/config identifiers, impact, and owner. Export logs only through approved access; preserve original timestamps/integrity and chain of custody where required.

14. Health checks

Use [Operations Runbook RB-03](#). A green endpoint alone is insufficient.

Layer · Safe check · Healthy evidence

DNS/TLS · Resolve approved domains; inspect certificate name/chain/expiry · Expected endpoints and trusted valid chain

Frontend · HTTPS homepage and representative static assets/package page · Expected HTTP result/checksum; no fallback presented as live

API · Bounded health endpoint through public boundary · Healthy response without internal disclosure

Boundary · NGINX config test and active service · Syntax valid, active, expected routes/headers/redirects

Backend · Service state and loopback health under approved access · Active, stable, correct release/config identity

Database · Approved readiness/query and backup/storage signal · Reachable, correct database/schema, capacity acceptable

Authentication · Controlled synthetic/non-production or approved production-safe validation · Login/session behavior without secret/log leakage

Business · Catalogue read, Manual Finance capability, authorised customer/Finance read paths · Correct authoritative data and ownership boundaries

Dependencies · SMTP, scanner, proof storage, outbox/scheduler · Available or controlled degraded status with owner

Do not use destructive/write acceptance in production without a separately approved test-data procedure. Cache-bust frontend checks and compare checksums; HTML is non-cacheable while accepted static asset policy may retain JavaScript/CSS/images for one day.

15. Scheduled tasks

Maintain a restricted scheduled-task register:

Task · Purpose · Schedule/time zone · Concurrency/idempotency · Last success/alert · Owner/runbook

Notification/outbox dispatch · Deliver committed notification intents · [approved value] · Prevent duplicate effect; stale-claim recovery · [monitor] · Notification Owner / RB-06

Review expiry/SLA work · Apply approved review timing rules · [policy-owned] · One legal transition; audit · [monitor] · Finance/Operations

Proof scan/retention · Scan and manage private proof lifecycle · [policy-owned] · Never open untrusted; preserve holds · [monitor] · Security/Platform

Database backup/PITR archive · Protect database recovery point · [RPO-derived] · Consistent/encrypted · [monitor] · Backup Owner / RB-04

File/config backup · Protect proof/config recovery · [RPO-derived] · Reconcile and encrypt · [monitor] · Backup Owner

Certificate renewal/check · Prevent TLS expiry · [provider/owner] · Safe key handling · [monitor] · Certificate Owner

Log rotation/retention · Maintain capacity and approved records · [policy-owned] · Preserve audit requirements · [monitor] · Security/Platform

Do not infer schedules from application behavior. Verify actual service timers, cron jobs, worker configuration, and external schedulers during handover without printing secrets. A missed or overlapping run follows RB-07; never launch a second copy until concurrency and prior outcome are known.

16. Security

Required controls

- **Least privilege:** individual accounts, role separation, approved elevation, time-bounded access where possible, periodic review, and backup operators. Administrator capability never grants Finance authority.

- **Secrets:** store outside Git in the approved secret manager/environment mechanism; protect JWT key paths and SMTP credentials; rotate under dual control; never display in logs or backups without approved encryption.

- **Firewall/network:** expose only approved HTTPS; keep backend/PostgreSQL direct listeners restricted/loopback; verify rules and denial from unauthorised networks.

- **TLS/certificates:** trusted chain, correct names, modern policy, automated expiry alert, protected private keys, tested renewal/recovery.

- **CORS/CSRF/cookies/JWT:** exact production origin, approved issuer/audience/key rotation, secure HttpOnly refresh cookies, CSRF contract, no tokens in browser storage/logs.

- **NGINX boundary:** restrict public API, Swagger/admin routes, methods, sizes/timeouts/headers according to accepted configuration; log normalised paths only.

- **Proof storage:** private, non-public, least privilege, encrypted/backup-controlled, isolated scanning, safe format/status checks, retention/legal-hold policy, no ordinary download/forwarding.

- **Audit:** immutable business/security evidence, UTC/time synchronisation, access control, monitoring, approved retention, and protected export.

- **Vulnerability/patching:** approved inventory, vendor advisories, risk review, maintenance/rollback, and verification; no ad hoc production upgrade.

Suspected compromise, malicious proof, credential exposure, wrong-customer data, or audit tampering requires immediate Security/Privacy escalation and containment under the incident plan.

17. Certificates

The certificate register must include domain names, issuing authority, serial/fingerprint, installation point, private-key owner/location reference (not value), renewal mechanism, expiry, alert lead time, validation method, revocation/reissue route, and backup contact.

Routine renewal:

1. Confirm approved renewal owner/tool/account and DNS/HTTP challenge dependency.
2. Verify system time, current certificate/chain/expiry, available capacity, and backup/config evidence.
3. Renew through the approved mechanism without exposing key material.
4. Validate configuration, reload only the affected boundary, verify both domains/chain/expiry, and monitor.
5. Record certificate fingerprint/expiry and close the change.

Expiry/compromise follows the DR guide. Never copy a private key into a ticket or use an unverified emergency certificate.

18. DNS

The DNS register must record zone/provider/registrant owners, multi-factor access/recovery, frontend/API records, TTL, change/rollback procedures, DNSSEC status where used, certificate challenges, and vendor support. Do not publish private network/management names in customer documentation.

For a change, lower TTL only when approved and sufficiently in advance, record previous/desired values, use peer verification, confirm authoritative and representative resolver results, validate TLS/application behavior, monitor, and restore intended TTL. DNS failure follows the DR guide; avoid simultaneous uncoordinated DNS and application changes.

19. NGINX

NGINX serves the static frontend and protects/routes the HTTPS API boundary. Accepted frontend releases use `/var/www/virtcruise/releases/<commit>/` and atomic `/var/www/virtcruise/current` selection. Release directories are read-only to NGINX (root:www-data, accepted baseline 0750 directories/0640 files).

Before any reload:

1. Back up/reference current approved configuration and active release.
2. Review change, routes, CORS/boundary impact, certificate, file paths, and rollback.
3. Run `sudo nginx -t`; stop on any warning/error outside approved baseline.
4. Reload NGINX only; do not restart unrelated services.
5. Verify HTTP-to-HTTPS, TLS, frontend, representative assets/routes, API health, headers/log safety, and direct-port restrictions.

The exact configuration repository/path and service unit must be confirmed in the restricted inventory. Do not rely on this guide as the configuration source.

20. PostgreSQL

PostgreSQL is authoritative for application persistence and Flyway schema history. Accepted evidence uses PostgreSQL 18 family with a loopback listener. Production exact version, database/role names, extensions, storage, backup/PITR, capacity, and service unit require restricted inventory verification.

Operational controls:

- use dedicated least-privilege application/backup/administration identities;
- restrict network access and protect credentials;
- monitor availability, connections, locks/long transactions, errors, storage/inodes, memory/CPU, backup/WAL age, and time;
- apply Flyway migrations only as part of an approved compatible release; record pre/post schema version;
- never edit business records or Flyway history to resolve an operational symptom;
- use logical/physical backup methods and point-in-time recovery appropriate to approved RPO/RTO;
- restore into isolation, validate integrity/schema/business reconciliation, then cut over under incident/change authority; and
- coordinate Financial data reconciliation with Finance/Operations before declaring recovery.

21. Application services

The restricted service catalogue records exact backend service unit, Java/runtime version, release path, configuration identifier, environment file/secret references, user/group, dependencies, resource limits, restart policy, health command, logs, and start/stop/reload sequence.

Safe service action:

1. Confirm incident/change authority, current release/config/schema and dependency health.
2. Assess in-flight requests, scheduled/outbox work, proof scan, customer/Finance impact, and idempotency.
3. Preserve logs/request IDs/state and communicate controlled degradation.
4. Use the approved service manager command; never kill/restart repeatedly without understanding failure.
5. Verify health, release/config identity, database/schema, queues/outbox, authentication, storage/scanner/SMTP, and logs.
6. Escalate recurrence; a restart is containment, not root-cause resolution.

22. Notifications

Notification business events and technical delivery are separate. Monitor pending, sending, retry, failed, cancelled, suppressed, uncertainty, and manual-intervention conditions only through the authoritative operations source supported by current production. The released Finance frontend has no notification-operations screen.

On delay/failure:

1. Verify the underlying booking/Payment/Receipt/review event independently.
2. Locate intent/attempt by safe reference and inspect authoritative state/time/owner.
3. Check dispatcher/service, SMTP provider, DNS/reputation where approved, queue age, retry policy, and recent changes.
4. Do not blindly resend, relabel FAILED as SENT, or bypass SUPPRESSED.
5. Assign Notification/Technical Operations and give Support a bounded customer update.
6. Recover under RB-06, verify no duplicate effective message, and record outcome.

SMTP credentials remain outside Git. Email outage should not trigger direct business-state changes.

23. Manual Finance operations

Production baseline v0.7.0 is MANUAL_FINANCE. Customers see configured, escaped Finance contact instructions, owned booking/invoice reference, currency, and explicit unpaid/unconfirmed wording. They do not see bank details, review creation, proof upload, or replacement controls.

Daily Operations/Finance checks:

- capability remains MANUAL_FINANCE on authorised customer paths;
- approved contact content, reference, and currency are present and safe;
- no customer-facing bank instruction/proof-upload controls appear;
- Finance contact route and handover are staffed under approved policy;
- Financial/booking outcomes remain authoritative and notifications are checked separately.

Switching to SELF_SERVICE is a backend configuration plus restart and a material business/security/Finance production change. It requires approved bank workbook/values, proof roots/scanner/storage/routes, operating readiness, customer/support communication, backups/rollback, release acceptance, and separate change authority. This guide does not authorise activation.

24. Support handover

Support Managers receive:

- current release/customer-visible change and known limitation summary;
- incident/maintenance status and approved customer wording;
- health/known-issue view that excludes secrets/internal proof/message content;
- approved routes for Finance, Operations, Technical Operations, Security/Privacy, and Management;
- expected customer references and safe evidence to collect;
- Manual Finance status and exact customer-safe boundary;
- next update owner/time when approved; and
- closure/recovery evidence and any customer follow-up population under privacy controls.

Support never diagnoses infrastructure to customers, promises recovery time without authority, or uses a notification/proof as authoritative booking/payment state. Follow DOC-005.

25. Incident response

Severity framework

Exact severity thresholds and response times are policy-owned. Use the approved incident policy. The following impact classes guide—not replace—that policy:

Impact class · Examples · Immediate coordination

Critical · Active security/privacy exposure; material integrity loss; widespread outage; unsafe Financial processing · Incident Commander, Security/Privacy, Platform, Business owner; contain immediately

High · Major customer/staff function unavailable; travel/financial impact increasing; recovery dependency failed · Operations, Platform/Technical, Support, affected domain lead

Moderate · Degraded/partial service with controlled workaround and no active integrity risk · Component owner and Operations; monitor/escalate trend

Low · Isolated low-impact defect or routine request · Normal queue/change process

Incident lifecycle

1. **Detect and verify:** record UTC time, reporter/signal, scope, affected component/release, safe symptoms, request IDs, customer/financial/travel impact.
2. **Declare and assign:** incident commander, severity under policy, technical/domain owners, communication owner, next review.
3. **Contain:** stop exposure/corruption/unsafe writes, isolate compromised component, freeze changes, preserve evidence; never destroy logs or guess.
4. **Diagnose:** compare dependencies, health, logs, recent changes, capacity, queues, backup state, and authoritative business records.
5. **Recover:** choose rollback, restart, vendor action, capacity relief, restore, or controlled degraded service under approved runbook/authority.
6. **Validate:** technical health, data/security/Financial reconciliation, customer journeys, monitoring, and absence of duplicate/out-of-order effects.

7. **Communicate:** factual impact/known action/owner/approved expectation; protect confidential detail.

8. **Close and learn:** closure decision, residual work, customer follow-up, evidence preservation, timeline, contributing controls, actions/owners/dates, postmortem.

Postmortems are blameless but accountable: distinguish facts from hypotheses, identify why controls detected/prevented or failed, and track corrective actions through change management.

26. Change management

Change type · Minimum control

Routine maintenance · Approved standard change/template, scope/risk, backup, validation, rollback, scheduled communication, record

Normal release · Candidate approval, artifact/schema/config evidence, full deployment/rollback checklists, peer execution, acceptance/observation

Emergency maintenance · Incident link, emergency authority, minimum safe peer/backup/rollback evidence, immediate validation, retrospective review

Security emergency · Security incident authority, containment/evidence, need-to-know communication, credential/key handling, later controlled remediation

Configuration/mode · Versioned non-secret change, dual verification, business/security owner, restart/impact, rollback; secrets remain managed

Database/data recovery · Incident/change authority, recovery point/data-loss assessment, isolated restore validation, domain reconciliation, controlled cutover

Every change record contains purpose, scope, owner, risk/impact, affected inventory/config/release, prerequisites, backup evidence, execution steps/runbook, approver, window, communications, monitoring, rollback triggers/target/authority, validation, actual outcome, evidence, incidents/deviations, and closure.

No verbal approval, urgent request, or visible control replaces the record. Emergency changes receive retrospective review and action follow-up.

27. Operational checklists

Production Checklists contains eight controlled checklists:

1. Deployment.
2. Rollback.
3. Backup.
4. Restore.
5. Daily operations.
6. Weekly maintenance.
7. Monthly maintenance.
8. Disaster recovery.

Checklists prompt and evidence a procedure; they do not replace runbooks, authority, or judgment. Record Yes, No, or Not applicable with evidence/reason. A blocking No stops the activity.

28. Known risks

Risk/assumption · Consequence · Required control/owner

Accepted topology appears single-host/loopback; redundancy/failover not evidenced · Host/site failure may affect frontend/API/database together · Platform Owner completes architecture/inventory and approves resilience/RTO

Complete live server/service/storage/backup inventory absent from this repository · Recovery depends on undocumented knowledge · Restricted inventory is a handover acceptance gate

RPO, RTO, retention, severity, alert, capacity, maintenance values are policy-owned/unrecorded here · Inconsistent or unsafe decisions · Business/Platform/Security owners approve and record values

Restore evidence and independent backup failure-domain not evidenced here · Backups may not be recoverable after shared failure · Backup Owner proves isolated restores and storage independence

Proof storage/ClamAV production roots/services not fully documented here · Unsafe/missing proof and Finance disruption · Security/Platform restricted inventory, alerts, backup/DR validation

SMTP appears external and credentials are environment-managed · Provider/account/DNS failure blocks messages · Notification owner documents provider recovery and safe degraded communication

Static assets may cache for one day · Mixed old/new client assets after release · Cache-busting/checksums; future fingerprinted assets

MANUAL_FINANCE depends on staffed approved contact route/configuration · Customers cannot self-serve payment proof · Finance/Support readiness and configuration monitoring

SELF_SERVICE is accepted conditionally but not production-approved · Premature activation exposes financial/security risk · Separate governed activation; no ops workaround

Accepted REJECTED case is terminal despite replacement design · Customer replacement journey unavailable after rejection · Do not promise; product/policy hardening

No released notification-operations Finance screen · Delivery recovery depends on separate monitoring · Authoritative operations source/runbook and trained owner

Release/backend/schema/config compatibility spans repositories · Partial rollback can create incompatibility · Unified release manifest and dependency matrix

Documentation contains historic paths/versions · Operators could use stale target · Release-specific record/inventory controls; never copy historic value blindly

29. Future improvements

Prioritise through approved change management:

1. Complete and govern the restricted configuration/asset/dependency/contact inventory.
2. Approve measurable service objectives, RPO/RTO, backup retention, severity, capacity, and alert thresholds.
3. Test documented isolated restores and full disaster-recovery tabletop/failover at a scheduled cadence.
4. Reduce single points of failure through proven redundancy/failover based on business impact analysis.
5. Establish unified frontend/backend/schema/config release manifests and signed/provenance-controlled artifacts.
6. Add fingerprinted static assets and explicit cache-version strategy.
7. Provide approved notification operations visibility and controlled recovery.
8. Harden/accept replacement proof lifecycle before any customer promise.
9. Automate certificate/DNS/backup/restore-age/capacity monitoring with actionable runbooks.
10. Maintain tested break-glass access, secret/key rotation, vendor recovery, and loss-of-admin procedures.
11. Create privacy-safe synthetic production acceptance and cleanup governance if business requires live release transactions.
12. Review the package after every material architecture, vendor, policy, security, or release-process change.

30. Appendices

Appendix A — Handover acceptance checklist

- [] Restricted inventory and contact/escalation registers complete and peer verified.
- [] Individual least-privilege and break-glass access tested without sharing credentials.
- [] Current/previous releases, checksums, schema, configuration identifiers, and rollback compatibility recorded.
- [] DNS, certificates, NGINX, service units, database, storage, SMTP, scanner, monitoring, logging, scheduled tasks, and vendors have owners/backups.
- [] Approved RPO/RTO, backup retention, severity, capacity, alert, and maintenance policy values recorded.
- [] Recent backups succeeded and isolated restore evidence is accepted.
- [] Deployment/rollback, incident, restore, and DR tabletop witnessed.
- [] Manual Finance/Support/Finance customer handover is ready.
- [] Known risks have owners, controls, due dates, and accepted residual risk.
- [] Receiving Platform Owner signs acceptance; unresolved blockers remain explicit.

Appendix B — Production handover record

Field · Entry

Handover scope/environment ·

Current frontend/backend/schema/config IDs ·

Inventory/contact register version/location ·

Backup/restore evidence and dates ·

Last deployment/rollback/DR exercise ·

Open incidents/changes/risks ·

Outgoing owner / receiving Platform Owner ·

Acceptance date / decision · Accepted / Conditional / Withheld

Conditions/outstanding owners/dates ·

Appendix C — Operational evidence record

Activity · Change/incident reference · Start/end UTC · Operator/verifier · Pre-state · Action/runbook · Validation · Outcome/follow-up

Appendix D — Policy-owned values register

Value · Policy owner · Approved value/reference · Review date · Operational consumer

RPO / RTO by service/data · Business/Platform · · · Backup/DR/incident

Backup retention/geography · Security/Privacy/Business · · · Backup/restore

Incident severity/response · Operations/Business · · · Alerts/incidents

Alert/capacity thresholds · Platform/Operations · · · Monitoring

Maintenance windows/approval · Change Owner · · · Releases/patching

Certificate renewal lead time · Security/Platform · · · TLS monitoring

Proof retention/legal hold · Security/Privacy/Finance · · · Storage/backup

Notification retry/suppression · Notification/Business · · · Outbox/Support

Finance SLA/expiry/contact · Finance · · · Manual Finance/reviews

Appendix E — Screenshot placeholder register

Figure · Subject · Views · Safe capture requirement/dependency

- 1 · Monitoring/service overview · Desktop, Tablet, Mobile · Synthetic/redacted labels; no hosts, IPs, tokens, customer data; approved monitoring view
- 2 · Release/rollback evidence · Desktop, Tablet · Fictional release IDs/checksums; approved change system
- 3 · Backup/restore dashboard · Desktop, Tablet · Fictional jobs/targets; no storage paths/credentials; approved backup system
- 4 · Incident command record · Desktop, Tablet, Mobile · Fictional incident/contacts; approved incident system
- 5 · Certificate/DNS register · Desktop, Tablet · Public sample domains or masked inventory; no private keys/recovery data

Screenshot required — Figures 1–5 only after controlled capture approval. No screenshot is fabricated. Follow DOC-001 screenshot standards and obtain Security/Privacy review.

Appendix F — Workflow diagrams

Figures 1–3 cover production ownership topology, release governance, and disaster recovery. The Disaster Recovery Guide adds recovery-priority and scenario decision flows; the Operations Runbook adds command-control flow summaries.

Related documents

- [Production Checklists](#)
- [Disaster Recovery Guide](#)
- [Operations Runbook](#)
- [DOC-003 Back Office Operations Manual](#)
- [DOC-004 Finance SOP](#)
- [DOC-005 Support Playbook](#)
- [DOC-007 Status & Lifecycle Reference](#)
- [DOC-008 Quick Start Pack](#)
- [DOC-009 Training Manual](#)
- [Production Beta Operations](#)
- [Frontend Deployment](#)
- [Frontend Operations](#)
- [Release Notes v0.7.0](#)
- [Frontend Rollback v0.7.0-rc4](#)
- [Architecture](#)

Change history

Version · Date · Author · Change · Status

0.8.0-draft.1 · 2026-08-03 · Documentation Lead · Initial authoritative production handover package · Draft